CHAPTER 6 — SOFTWARE METRICS

Applied Scenarios & Architectural Evaluation

Section 1: Estimation Scenarios & Project Risk Analysis

Q6.S01. Scenario: Analyzing the Operational and Financial Consequences of Diverging Size Estimates on Project Success (Exercise 6.24).

Scenario Context: During the preliminary bidding phase of an enterprise enterprise resource planning (ERP) system, three senior software engineers independently estimate the source code size: - Engineer A (Optimistic): Estimates $30\ \mathrm{KLOC}$. - Engineer B (Most Likely): Estimates $60\ \mathrm{KLOC}$. - Engineer C (Pessimistic): Estimates $120\ \mathrm{KLOC}$.

Analysis of Consequences for Project Planning & Execution:

1. **Budgetary and Staffing Vulnerability:**
2. Using Basic COCOMO for a semi-detached system ($Effort = 3.0 \times (KLOC)^{1.12}$):
    - At $30\ \mathrm{KLOC}$, estimated effort is $\approx 134\ \mathrm{Person\text{-}Months\ (PM)}$.
    - At $60\ \mathrm{KLOC}$, estimated effort is $\approx 292\ \mathrm{PM}$.
    - At $120\ \mathrm{KLOC}$, estimated effort is $\approx 634\ \mathrm{PM}$.
3. The estimate range creates material planning uncertainty. If the low estimate is used as the sole planning basis, the project may be under-resourced; the scenario does not provide enough information to predict a specific financial outcome.
4. **Schedule and Milestone Risk:**
5. The size estimates alone do not determine a delivery schedule. Any schedule forecast also needs assumptions about staffing, productivity, dependencies, and model calibration.
6. **Other Planning Risks:**
7. If estimates are treated as commitments without reviewing assumptions, teams may face pressure to reduce scope or quality activities. These are possible consequences, not outcomes determined by the size numbers.

Engineering Decision & Risk Mitigation Strategy: - A three-point estimate can summarize the supplied range:

$$KLOC_{PERT} = \frac{KLOC_{opt} + 4 \times KLOC_{likely} + KLOC_{pess}}{6} = \frac{30 + 4(60) + 120}{6} = 65\ \mathrm{KLOC}$$

- The range-based standard deviation estimate is $\sigma = (120 - 30)/6 = 15\ \mathrm{KLOC}$ under the PERT approximation.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 140–149; companion slides Ch.4 & Ch.6. Exercise 6.24.

Q6.S02. Scenario: Selecting Appropriate Sizing and Estimation Techniques Across Differing Project Domains (Exercise 6.25).

Scenario Context: An engineering director must select the most appropriate software sizing technique for three upcoming development contracts: - **Project Alpha:** A high-transaction, data-intensive Cloud Retail Banking System specified via detailed user functional requirements and database entity models. - **Project Beta:** An Embedded Real-Time Avionics Flight Control Computer specified with complex differential equations and signal processing loops, but minimal user interfaces. - **Project Gamma:** An early-stage exploratory Rapid Mobile Prototyping project where requirements are volatile and interface mockups change weekly.

Engineering Evaluation & Technique Selection:

- **1. Recommendation for Project Alpha: Albrecht's Function Points (FP):**
- **Justification:** Banking systems are characterized by large volumes of transactional data inputs (EI), report outputs (EO), customer balance inquiries (EQ), and internal relational databases (ILF). Function Points quantifies this business utility accurately, early in the lifecycle, independent of implementation language.
- **2. Recommendation for Project Beta: Feature Points or McCabe/Halstead-Calibrated LOC:**
- **Justification:** Function Points may not capture all algorithmic work in this example. Feature Points explicitly includes an algorithm component; LOC is another possible size measure if its counting rules and intended use are stated.
- **3. Recommendation for Project Gamma: Object Points (Application Points):**
- **Justification:** In rapid prototyping, full function point transactions and detailed code are not yet finalized. Sizing based on countable high-level prototypes—**screens, reports, and 3GL components** (classified as simple, medium, or difficult)—provides rapid, lightweight size estimates aligned with COCOMO II Early Prototyping.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 142–148; Mall, R., 4th ed., printed pp. 165–166; Pressman, 5th ed. Exercise 6.25.

Section 2: Metric Interpretation & Architectural Diagnostics

Q6.S03. Scenario: Architectural Triage of Information Flow Hotspots (Fan-in and Fan-out Extremes).

Scenario Context: Static analysis of a legacy telecommunications billing gateway reveals two architectural anomalies: - **Module AuditLogger:** $fan_in = 68$, $fan_out = 1$. - **Module TransactionDispatcher:** $fan_in = 1$, $fan_out = 34$.

Cautious interpretation (no universal cutoffs are supplied):

1. **Diagnosis for AuditLogger (High Fan-in, Low Fan-out):**
2. **Interpretation:** Indicates high structural reuse. Dozens of components throughout the system rely on this centralized utility.
3. **Potential Concern:** Concentrated dependency can increase change impact. Inspect the actual dependency graph and tests to see which callers could be affected.
4. **Action Plan:** Identify affected callers and prioritize tests for important dependencies; check concurrency and I/O only where those behaviors are part of the requirements.
5. **Diagnosis for TransactionDispatcher (Low Fan-in, High Fan-out):**
6. **Interpretation:** May indicate broad coordination responsibility; the call graph alone does not prove a “God Object” or poor design.
7. **Potential Concern:** A broad call-out set may make testing and responsibility boundaries harder to understand; verify against the actual module design and tests.
8. **Action Plan:** Inspect the call graph and responsibilities; if the component combines separable roles, evaluate decomposition (for example, into coordinating controllers). The fan-out value alone does not require a particular design pattern or prove a defect.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), companion slides 69–77; Pressman 5e §19.4.2.

Q6.S04. Scenario: Use cyclomatic complexity to guide a code-review discussion.

Scenario: A CI report gives $V(G) = 6$ for calculateTaxRate(), 18 for parseHttpHeader(), and 54 for processInsuranceClaim().

$V(G)$ is the number of linearly independent basis paths in a control-flow graph. The value 6 indicates a basis set of six paths to identify and test; it does **not** automatically guarantee complete condition coverage. The values 18 and 54 indicate progressively larger basis sets and justify closer review of control structure and test design. The team may set its own CI thresholds, but the supplied course source does not establish universal cutoffs, defect probabilities, or an automatic merge-block rule. Any such policy should be stated as a project decision and calibrated to the system.

Practical response: inspect the CFG and existing tests for each function; identify independent paths and high-risk decisions; consider refactoring when the structure is hard to understand or test. Do not infer that a function with $V(G) = 54$ is untestable or certain to contain defects from the metric alone.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 422–425. CI actions and scenario values are illustrative; no universal threshold is supplied by this source.

Q6.S05. Scenario: Diagnosing Object-Oriented Structural Decay Using the Chidamber & Kemerer (CK) Metric Suite.

Scenario Context: Architectural auditing of a Java enterprise banking application generates the following CK metric profile for a core domain class: - Class CustomerAccountManager: - $WMC = 48$ - $DIT = 6$ - $NOC = 0$ - $CBO = 19$ - $RFC = 73$ - $LCOM = 86$

Architectural Diagnostics:

1. **$WMC = 48$ and $RFC = 73$: **
2. These are values under the chosen definitions. Without a project baseline or method-level details, they do not by themselves prove excessive complexity or a "God Class".
3. **$DIT = 6$: **
4. The class is six levels from the root under the stated convention. Review inherited behavior; the value alone does not prove a fragile-base-class problem.
5. **$CBO = 19$: **
6. The class is coupled to 19 other classes under the scenario's counting rule. This identifies dependencies to inspect; it does not make isolated testing impossible.
7. **Low Cohesion $LCOM = 86$:**
8. A high value under the selected LCOM variant can indicate limited attribute sharing among method pairs. Inspect the actual methods and responsibilities before concluding that the class has unrelated responsibilities or needs a split.

Next step: Inspect the methods, shared data, call relationships, and inheritance chain. Consider extraction or a hierarchy change only if that inspection confirms separate responsibilities or costly dependencies.

Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 288–294; companion slides 80–85.

Section 3: Organizational Metrics Programs, Pitfalls & Ethics

Q6.S06. Scenario: Analyzing behavior risks when software metrics become incentive targets.

Scenario Context: To accelerate delivery and enforce quality, executive management introduces an incentive bonus scheme: - Policy 1: A monthly monetary bonus is awarded to developers who produce the highest Lines of Code (LOC). - Policy 2: Individual performance reviews are tied to the number of defect tickets marked "Resolved" per week.

Possible behavior risks in this scenario:

1. **Dysfunction Under Policy 1 (Bonus for LOC):**
2. **Possible response:** Some developers may add unnecessary code or avoid reuse to increase the measured LOC.
3. **Risk:** LOC may rise without a corresponding increase in delivered function or quality.
4. **Dysfunction Under Policy 2 (Performance Tied to Closed Defects):**
5. **Possible response:** Developers may prioritize easy tickets to increase closure counts, while difficult work receives less attention.
6. **Risk:** A higher closure count may not mean that overall defect risk has fallen.

Review response: Check that the measure matches the intended outcome, explain how it will be used, and interpret it with other relevant evidence and context. Professional obligations include public interest, product quality, and sound professional judgment.

Source: Sommerville, 9th ed., §1.2, printed pp. 14–16. The scenario and responses are authored illustrations.

Q6.S07. Scenario: Statistical Analysis of Metric Data: Evaluating Correlation versus Causation (Exercise 6.18).

> **Scenario Context:** A software metrics analyst collects empirical data across 50 production modules and observes a strong positive correlation ($r = 0.88, p < 0.001$) between **Lines of Code (LOC)** and the **Number of Post-Release Defects**.
>
> Based on this correlation, the engineering director issues a policy: "*To cut our defect rates in half, all developers must strictly limit module sizes to under 200 lines by arbitrarily splitting code.*"
>
> **Critical Methodological Evaluation:**
>
> 1. **The Fallacy of Confusing Correlation with Causation:**
> 2. Correlation ($r = 0.88$) establishes mutual statistical association, **not direct causation**. Both LOC and defect count are driven by an underlying confounding variable: **functional domain complexity and problem size**.
> 3. A complex accounting calculation requires extensive logic; the sheer volume of business rules naturally increases both the required code length and the statistical opportunity for human error.
> 4. **Negative Architectural Consequences of the Arbitrary Slicing Policy:**
> 5. Artificially splitting a cohesive 400-line algorithm into three arbitrary 150-line sub-functions does not reduce problem complexity; instead, it **increases inter-module coupling (CBO, fan-out), scatters logic across files, damages cohesion, and introduces new interface and parameter-passing defects**.
> 6. **Sound Statistical Action Plan:**
> 7. Perform **multivariate regression analysis** controlling for domain complexity, cyclomatic complexity ($V(G)$), developer experience, and test coverage.
> 8. Focus on **Defect Density ($Defects/KLOC$)** rather than raw defect counts to evaluate genuine process quality objectively.
>
> *Source: K.K. Aggarwal & Yogesh Singh (3rd ed.), printed pp. 296–300; companion slides 92–95. Exercise 6.18.*

Explain why under-estimating project size by a factor of two increases COCOMO effort by more than double.
Differentiate between appropriate sizing metrics for a banking system, an avionics controller, and an early mobile prototype.
Outline the architectural triage for a module with $fan_in = 68$ versus one with $fan_out = 34$.
Explain what the three example $V(G)$ values establish about basis paths, and distinguish source-supported interpretation from a team-defined CI policy.
Describe possible behavior risks when developer compensation is tied to LOC or closed-ticket counts, without treating them as guaranteed effects.